Computational Methods #6

CH5 EX6

The goal of this exercise is to compute the inner product matrix of the set of polynomials

$$\phi_n = \frac{\pi^{-1/4}}{\sqrt{2^n n!}} \exp\left(-x^2/2\right) H_n\left(x\right)$$

We will need to loop over the elements of the inner product matrix $\langle\phi_m|\phi_n\rangle$ and populate them by evaluating the integrals. We can either generate each function in the same loop as the integration or generate all functions first and then call them during integration, I chose the latter. The funcitons are generated with the following block of code

```
//Implementing the basis functions into an array

std::vector<std::unique_ptr<Genfun::AbsFunction>> phiList;
for(int n=0; n<Nbasis; n++){
  GENFUNCTION phi = pow(M_PI,-0.25)/(sqrt(pow(2,n)*factorial(n)))
                      *HermitePolynomial(n)*exp(-X*X/2);
  phiList.push_back(std::unique_ptr<Genfun::AbsFunction>(phi.clone()));
}
```

In each iteration a `GENFUNCTION` phi is created and a pointer to it is added to `phiList`. It must be cloned first since `phi` ceases to exist outside the loop so we point to a copy.

The second step is numerical integration with a Gaussian quadrature using an instance of `GaussIntegrator` on the functions pointed to by `phiList`.

```
Eigen::MatrixXd innerProductMatrix = Eigen::MatrixXd::Zero(Nbasis,Nbasis);
for (int m=0; m<Nbasis; m++){
 for (int n=0; n<Nbasis; n++){
   innerProductMatrix(m,n) = integrator((*phiList[n])*(*phiList[m]));
 }
}
```

In the main function we see that the inner product matrix is the identity matrix if the number of integration points is equal to or greater than the number of basis functions used.

CH5 EX7

Using the list `phiList` we can produce generate the operators in this basis representation. These have simple analytical solutions by representing the non-dimensionalized position and momentum operators in terms of raising and lowering operators $x = a^\dagger + a$ and $p = a^\dagger - a$ and we get the familiar result that x and p only have super and sub diagonal elements. The matrix representation of H comes out to be a diagonal matrix with $\langle n| H |n\rangle = n + 1/2$ as expected.

```
[x] =
-1.04083e-17      0.707107             0             0             0
    0.707107  -2.77556e-17             1             0  3.33067e-16
           0             1             0       1.22474             0
           0             0       1.22474             0       1.41421
           0  3.33067e-16             0       1.41421             0
[x^2] =
         0.5 2.77556e-17     0.707107             0 2.22045e-16
2.77556e-17           1.5             0       1.22474             0
   0.707107             0           2.5             0       1.73205
           0       1.22474             0           3.5             0
2.22045e-16             0       1.73205             0             2
[d/dx] =
 1.38778e-17      0.707107 -6.93889e-18 -9.98767e-16             0
   -0.707107             0             1 -5.37764e-17  1.11022e-16
           0            -1             0       1.22474             0
           0             0      -1.22474 -1.73472e-17       1.41421
           0  3.88578e-16 -5.55112e-17      -1.41421             0
[H] =
          0.5  1.38778e-17 -6.10623e-16  1.38778e-17  1.42941e-15
 1.38778e-17           1.5  1.38778e-17  4.44089e-16             0
-1.66533e-16             0           2.5             0             0
           0  4.44089e-16             0           3.5             0
   2.498e-16 -2.77556e-17 -1.11022e-16  5.55112e-17           4.5
```

CH5 EX9

To find the bound states of the system of two particles interacting through the charmonium potentials,

$$V(r) = -\alpha \frac{4\hbar c}{3r} + \frac{r}{\hbar c a^2}$$

We must find the roots of the equation $J(E_{n,\ell}) = \hbar(n + 1/2)$. Where the J is the action given by

$$\hbar\pi(n + 1/2) = \int_{r_{\min}}^{r_{\max}} dr \sqrt{2\mu(E_{n,\ell} - V_{\text{eff}}(r))}$$

Where μ is the reduced mass $\mu = m_c/2$, $r_{\min}$ and $r_{\max}$ are the turning points in the effective potential $V_{\text{eff}}(r)$ which is given by,

$$V_{\text{eff}}^{\ell}(r) = -\alpha \frac{4\hbar c}{3r} + \frac{r}{\hbar c a^2} + \frac{\ell(\ell+1)\hbar^2}{m_c r^2}$$

We can simplify this equation by using $x = r/\hbar c$ as the redimensionalized length which will have units MeV^{-1}.

$$V_{\text{eff}}^{\ell}(x) = \frac{1}{a^2}x - \frac{4\alpha}{3}\frac{1}{x} + \frac{\ell(\ell+1)}{m_c c^2}\frac{1}{x^2}$$

To solve for $E_{n,\ell}$ first we need a function which returns the turning points given the energy $x_{\min}(E)$ and $x_{\max}(E)$ by solving $V_{\text{eff}}^{\ell}(x_{\text{turn}}) = E$. We can multiply this equation by x^2 and solve the cubic polynomial using a companion matrix

```
std::vector<double> getTurningPointsCompanion(double energy, double l){
  double alpha{0.38};
  double mc2{1370};
  double a{2.43e-3};

  double c0 = (l+0.5)*(l+0.5) * a*a / mc2;
  double c1 = -4.0 * alpha * a*a /3.0;
  double c2 = -energy * a*a;

  Eigen::Matrix3d CompanionMatrix{{0.0, 0.0, -c0},
                                  {1.0, 0.0, -c1},
                                  {0.0,1.0, -c2}};

  Eigen::EigenSolver<Eigen::Matrix3d> solver(CompanionMatrix);
  Eigen::Vector3d eigenvalues = solver.eigenvalues().array().real();
  std::sort(eigenvalues.begin(), eigenvalues.end());
  double xmin = eigenvalues[1];
  double xmax = eigenvalues[2];

  return {xmin, xmax};
}
```

Once we have the functions $x_{\min}(E)$ and $x_{\max}(E)$ we can build the function $J(E,\ell)$ by perfoming the integral

$$J(E,\ell)/\pi\hbar = \frac{1}{\pi}\int_{x_{\min}}^{x_{\max}} dx\,\sqrt{m_c c^2\left(E_{n,\ell} - V_{\text{eff}}^{\ell}(x)\right)}$$

```
double action(double energy, double l){
  double alpha{0.38};
  double mc2{1370};
  double a{2.43e-3};

  std::vector<double> turningPoints = getTurningPointsCompanion(energy,l);

  Genfun::Variable X;
  Genfun::GENFUNCTION sqrt = Genfun::Sqrt();

  Genfun::GENFUNCTION integrand=(1/M_PI) * sqrt(mc2) * sqrt(energy - X/(a*a)
                          + 4*alpha/3 /X - (l+0.5)*(l+0.5) /mc2 /(X*X));
  Genfun::SimpleIntegrator integrator(turningPoints[0],turningPoints[1]
                                      ,Genfun::OpenTrapezoidRule(), 1000);

  return (integrator(integrand)-0.5);
}

template<int L>
double action(double energy) {return action(energy, L);}
```

Then find the energy by simply root finding $J(E,\ell)/\pi\hbar = n + 1/2$. This generates the output,

```
1S: E = 340.541 MeV , Mass = 3080.54
2S: E = 931.991 MeV , Mass = 3671.99
3S: E = 1367.12 MeV , Mass = 4107.12
1P: E = 758.815 MeV , Mass = 3498.81
2P: E = 1214.38 MeV , Mass = 3954.38
3P: E = 1598.68 MeV , Mass = 4338.68
```

CH6 EX4[1]

The implementation for the StokesVector class and the Mueller Matrix class are in `StokesVector.cpp` and `MuellerMatrix.cpp`. They are based on the classes `JonesVector` and `JonesMatrix` classes provided in the example code. The main difference is that I made the member data an `Eigen::Vector4d` for a Stokes vector and a `Eigen::Matrix4d` for a Mueller matrix to simplify all the operations in their methods. In the main file I demonstrate converting a horizonatally polarized beam to a vertically polarized one using a half wave plate and polarizing a horizontally polarized beam to a diagonally polarized beam.

[1] I used Claude a few times while making this when I got stuck